SEO Audit Workflow — End-to-End (Slack → DataForSEO → Claude → Google Sheets → Slack)

This PDF describes an implementation-ready workflow to run a slash-command triggered SEO audit using Slack, DataForSEO APIs, Claude for analysis/text generation, Google Sheets for reporting, and MongoDB for logs/config. It documents: inputs, Slack payload and scopes, dynamic configuration, the DataForSEO API calls you listed, which responses should be summarized for Claude, Google Sheets tab structure (columns & formulas), how to create & share the Sheet, and how to return the result back to Slack.

1. High-level Flow

1. User invokes Slack slash command: `/seo-audit "site-url"` 2. Your backend (Bolt or custom server) receives payload (user_id, text, channel, response_url). 3. Backend fetches user email via `users.info` (requires `users:read.email` scope). 4. Backend loads global config from MongoDB / master Google Sheet. 5. Backend calls DataForSEO APIs (domain metrics, ranked keywords, backlinks, competitors, domain intersection, search volume, serp tasks, on-page parsing). 6. Backend stores raw JSON / task_ids in MongoDB and polls task_get endpoints for results. 7. Curated summaries are sent to Claude for analysis and to generate actionable outputs (priority tasks, schema, implementation steps). 8. Backend writes raw data + Claude outputs into Google Sheets (multiple tabs) using Google Sheets API and creates any formulas required. 9. Drive API shares the sheet with the requesting user's email. 10. Slack `response_url` or `chat.postMessage` is used to notify user and deliver the Sheet link.

2. Slack Input / Payload (Exact Fields)

Slack slash command POST body includes (example JSON / url-encoded): - token (verification token) - team_id, team_domain - channel_id, channel_name - user_id, user_name - command (e.g. /seo-audit) - text (the argument; expected: target site URL) - response_url (for async messages) - trigger_id
Important Slack OAuth scopes your app will need:
- commands (to receive slash commands) - users:read.email (to fetch user email for sharing sheets) - chat:write (to post messages) - files:write (if uploading files to Slack) - im:write or chat:write for DMs

3. Getting the user email (example)

- After receiving the slash payload, call Slack Web API `users.info` with `user_id`. - Example (Node Bolt): `await client.users.info({ user: user_id });` → response.user.profile.email - Ensure `users:read.email` is approved in OAuth scopes; otherwise you'll get no email.

4. Dynamic Configuration Strategy

- Global defaults (stored in master Google Sheet or MongoDB) - Brand-specific overrides (detect from domain; e.g., Honeypack vs Blackball) - Dynamic discovery: competitors via DataForSEO `/competitors_domain` API - User overrides via Slack modal or flags in the slash command

5. DataForSEO API Calls (phase mapping to your doc)

Phase 1 - Domain Metrics (your site)	POST /dataforseo_labs/google/domain_metrics/task_pos
Phase 1 - Ranked Keywords (your site)	POST /dataforseo_labs/google/ranked_keywords/task_p
Phase 1 - Backlinks Summary (your site)	POST /backlinks/summary/task_post
Phase 2 - Competitors (discover)	POST /dataforseo_labs/google/competitors_domain/task
Phase 3 - Competitor Domain Metrics	POST /dataforseo_labs/google/domain_metrics/task_pos
Phase 3 - Competitor Ranked Keywords	POST /dataforseo_labs/google/ranked_keywords/task_p
Phase 3 - Competitor Backlinks Summary	POST /backlinks/summary/task_post

Phase 3b - Competitor Backlinks List	POST /backlinks/backlinks/task_post
Phase 4 - Content Gap (domain_intersection)	POST /dataforseo_labs/google/domain_intersection/task
Phase 4 - Keywords from Site	POST /keywords_data/google_ads/keywords_for_site/ta
Phase 5 - Search Volume	POST /keywords_data/google_ads/search_volume/task_
Phase 5 - Keyword Suggestions	POST /keywords_data/google_ads/keywords_for_keywo
Phase 6 - SERP Analysis (organic)	POST /serp/google/organic/task_post
Phase 7 - Content Parsing (instant)	POST /on_page/instant_pages

6. What to store in MongoDB (recommended)

- Original slash request metadata (user_id, team_id, text, response_url) - Audit run id / job id - DataForSEO task IDs and raw JSON responses (or links to storage) - Claude prompts & responses (for auditing) - Google Sheet ID and sharing metadata - Execution timestamps, errors, and status

7. What to send to Claude (curation guidance)

Send **curated summaries** — do not send entire raw JSON blobs. For each DataForSEO response prepare a concise summary payload for Claude: 1) Domain Metrics Summary: site, traffic estimate, keywords_count, referring_domains_count, top growth/decline indicators. 2) Ranked Keywords Snapshot: top 50–100 keywords (keyword, position, volume, CPC, competition, URL). 3) Backlinks Summary: total backlinks, referring domains, dofollow ratio, top linking domains (top 10). 4) Competitors Summary: list top 3 competitors with their traffic, keywords_count, referring_domains_count. 5) Content Gap List: top 50 gap keywords (keyword, competitor_rank, search_volume, cpc, competition). 6) SERP Features Summary: for each opportunity keyword, include top 10 rankers, SERP features present (snippet, PAA, etc.). 7) On-page issues: top 10 issues per page (missing meta, missing schema, H1, etc.). 8) Config context: brand (Honeypack/Blackball), compliance flags, and audit depth.

8. Example JSON snippet to send to Claude (trimmed)

```
{
  "site": "mywebsite.com",
  "domain_metrics": {"traffic": 12345, "keywords": 2345, "ref_domains": 120},
  "top_keywords": [
    {"keyword":"best honey jar","position":12,"volume":2400,"cpc":1.25},
    {"keyword":"raw honey uses","position":5,"volume":5400,"cpc":0.85}
  ],
  "content_gaps": [
    {"keyword":"honey benefits for skin","competitor":"competitor1.com","rank":3,"volume":6600,"cpc
  ],
  "serp_features":[ {"keyword":"honey benefits","has_snippet":true,"snippet_holder":"competitor2.co
  "brand":"Honeypack",
  "compliance":{"requires_fda_disclaimer":true}
}
```

9. What Claude should return (examples)

- Competitive Intelligence narrative (3–5 bullet points) - Priority Action Items: table with task, impact (1-5), effort (1-5), recommended owner - Implementation Guide: step-by-step tasks (for developers/SEO) - Copy-paste ready HTML/JSON-LD schema snippets (for Technical Fixes tab) - Content briefs/outlines for top 10 opportunity keywords

10. Google Sheets Structure (tabs & intended contents)

1. Domain Overview	Site vs competitors: traffic, keywords_count, referring_domains. Columns: Site, Traffic

2. Content Gap	Keywords competitors rank for but you don't. Columns: Keyword, CompDomain, Com
3. Top Keywords	Your current rankings. Columns: Keyword, Position, Volume, EstTraffic, URL, Striking
4. Backlink Gap	Domains linking to competitors but not you. Columns: LinkingDomain, TargetComp, D
5. SERP Features	Keywords with snippets, PAA, reviews. Columns: Keyword, HasSnippet, SnippetHold
6. Technical Fixes	Claude-generated HTML/schema + code snippets (copy-paste)
7. Competitive Intelligence Matrix	Narrative + summary metrics per competitor
8. Priority Action Items	Task list with Impact/Effort/Score (Impact/Effort)
9. Implementation Guide	Step-by-step developer tasks
10. Cross-Brand Strategy	Honeypack vs Blackball differentiation
11. Performance Tracking	Baseline metrics and sparklines/trends

11. Sample Google Sheets formulas (practical examples)

```
- Priority score (Content Gap): =IF(C2>0,(C2*D2)/E2, C2/E2)  -- where C=Volume, D=CPC, E=Competitio
- Priority flag: =IF(G2>50000,"HIGH",IF(G2>10000,"MEDIUM","LOW"))  -- where G=Score
- Striking distance: =IF(AND(Position>=11,Position<=20),"■ Striking Distance","")
- Competitor backlinks multiple: =IF(B2>0, ROUND(C2/B2,1), "—")  -- where B=YourRefDomains, C=CompR
- Dofollow ratio check: =IF(DofollowRatio<0.7,"■■ Low dofollow ratio","OK")
- Alert: More than 50 keywords dropping: =IF(CountKeywordsDropping>50,"■■ More than 50 keywords dr
- Estimated monthly clicks (basic): =Volume * CTR_by_Position(Position) -- implement CTR table look
```

12. Writing Claude results into Google Sheets (pattern)

1) Prepare a JSON-to-Sheet mapping on the backend. Example mapping: - claude_response.priority_items -> write into 'Priority Action Items' sheet (columns: task, impact, effort, owner, notes) - claude_response.technical_snippets -> write into 'Technical Fixes' (columns: file/context, snippet) - claude_response.content_briefs -> write into 'Content Gap' or 'Top Keywords' as needed. 2) Use Google Sheets API `spreadsheets.values.batchUpdate` to write ranges in a single request. 3) For richer blocks (HTML/JSON-LD), write as plain text in cells with 'Wrap' enabled or to a dedicated cell per snippet. 4) Store Claude raw response in MongoDB for traceability and for reproducing or re-generating outputs.

13. Sharing the Sheet & Slack Notification

- Create the sheet via Google Sheets API (service account or project account). - Use Drive API to `permissions.create` -> share with requesting user's email (from Slack). - Optionally set domain-wide sharing if workspace integration allows. - Post message to Slack with the link using `chat.postMessage` or `response_url`. Example message: '■ SEO Audit complete for *mywebsite.com*. Report: '

14. Logging & Error Handling

- Log all DataForSEO task_ids and their status. Poll task_get until status=ready or timeout. - If an API fails, write error to MongoDB and post an ephemeral Slack message to the requestor with next steps. - Keep Claude prompts & outputs in logs for compliance and reproducibility.

15. Example Implementation Checklist (developer handoff)

• Create Slack App & add slash command + required OAuth scopes
• Implement slash command endpoint and verify payload
• Call users.info to fetch user email (users:read.email)
• Build job orchestration to call DataForSEO APIs & store task_ids
• Poll DataForSEO task_get endpoints and store results
• Transform & curate summaries for Claude inputs

• Call Claude API for analysis & text/code generation
• Write raw + processed data to Google Sheets with proper ranges & formulas
• Share sheet via Drive API to user email
• Send Slack notification with sheet link and store execution logs in MongoDB

16. Appendix: Minimal Example Slack handler pseudocode

```
/* PSEUDOCODE (Node.js with Bolt)
app.command('/seo-audit', async ({ ack, body, client }) => {
  await ack();
  const site = body.text; const user_id = body.user_id;
  // fetch email
  const user = await client.users.info({ user: user_id });
  const email = user.user.profile.email;
  // create job in MongoDB with status=queued
  // call DataForSEO domain_metrics task_post for site -> store task_id
  // call other DataForSEO tasks (ranked_keywords, backlinks,...)
  // poll task_get endpoints until ready; store results
  // prepare summaries & call Claude for analysis
  // create Google Sheet and write data + formulas
  // share sheet with email via Drive API
  // notify user with chat.postMessage or response_url
});
```

End of document — Generated by ChatGPT (workflow spec).